1 Real-time SSR corrections

The real-time engine consumes network corrections in *State Space Representation* (SSR): per-satellite, per-signal refinements to the broadcast orbit, clock and hardware biases. They arrive as RTCM 3.x messages (legacy 1057–1068 and 1240–1270) or in the IGS-SSR `4076` repackaging [1, 2]. Each SSR quantity refines one term already present in the undifferenced code equation and the undifferenced phase equation of the Foundations document, and none introduces a new state. SSR is what distinguishes the precise real-time mode from broadcast-only operation: with SSR applied, $c\,\mathrm{d}t^i$ and $\boldsymbol{r}^i$ are centimetre-grade, and the satellite biases B_s^i, b_s^i are *known* instead of absorbed into the ambiguity. This document describes the rover-side *consumer*. The same store is the sink for a third-party IGS-RTS stream and for the engine's own network estimator (the Network mode document), so the application mathematics below does not depend on the producer.

Decoded records are cached per satellite in `products/SsrCorrectionStore`: a process-wide singleton for the live stream, or a caller-owned instance for the network producer's closed-loop test. The consumers (`model/ModelGeometry`, `corrections/GnssCorrections`, `measurement/MeasurementModel`) query the store and fall back to broadcast-only when it is empty. In post-processing the store is empty, so every equation below reduces to the broadcast-only observation model. The streaming and decoding layer (RTCM frame parsing, the per-satellite mutex-guarded cache, the real-time push loop) is described in the Real-time engine document. This document covers only the store-driven correction mathematics, which does not depend on how the records arrived. There is no `enable_ssr` flag: SSR is applied whenever the store holds a usable record.

1.1 Orbit correction: refines ρ_r^i

The orbit message carries a delta in the satellite's radial, along-track and cross-track (RAC) frame, referenced to the broadcast ephemeris of issue IODE. The frame is built from the broadcast position $\boldsymbol{r}^i_{\mathrm{brdc}}$ and velocity $\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}$ (`products/SsrCorrectionStore.cpp`):

$$\hat{\boldsymbol{e}}_a = \frac{\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_c = \frac{\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_r = \hat{\boldsymbol{e}}_a \times \hat{\boldsymbol{e}}_c. \tag{1}$$

The RAC delta is propagated to the query epoch with its rate term and rotated into ECEF. With $\delta t = t - t_{\mathrm{SSR}}$ the age of the correction, $\boldsymbol{\delta}_{\mathrm{rac}} = (\delta_r, \delta_a, \delta_c)^{\mathrm{T}}$ its components and $\dot{\boldsymbol{\delta}}_{\mathrm{rac}}$ their rate,

$$\boxed{\boldsymbol{r}^i \;=\; \boldsymbol{r}^i_{\mathrm{brdc}} \;-\; \big[\,\hat{\boldsymbol{e}}_r \;\; \hat{\boldsymbol{e}}_a \;\; \hat{\boldsymbol{e}}_c\,\big]\big(\boldsymbol{\delta}_{\mathrm{rac}} + \dot{\boldsymbol{\delta}}_{\mathrm{rac}}\,\delta t\big).} \tag{2}$$

The subtraction follows the RTCM sign convention $\boldsymbol{\delta}$ = broadcast − precise (`products/SsrCorrectionStore.cpp`). If the broadcast velocity or the position-velocity cross product is degenerate, the frame is undefined and the broadcast position is returned unchanged. The corrected $\boldsymbol{r}^i$ enters $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line-of-sight $\boldsymbol{e}^i_{\hat{r}}$ of the Foundations document unchanged. The orbit correction therefore acts on no observation equation directly, only on the geometry (the observation-geometry section) that feeds it.

The delta is applied only when its IODE matches that of the broadcast ephemeris from which the baseline orbit was evaluated (Equation (6)). On a live ephemeris cutover, the baseline position is re-referenced to the ephemeris *version* against which the SSR delta was issued, so the satellite does not drop to broadcast grade (`model/ModelGeometry.cpp`, via `LiveEphemerisStore:`

`:computeSatellitePV` with the SSR IODE). The clock is queried against the same broadcast ephemeris, so the broadcast clock's *radial* component cancels consistently against Equation (4). The decoded RAC scale factors are those of RTCM 10403.3: the radial δ_r is a 22-bit two's-complement field at 0.1 mm, along-track and cross-track are at 0.4 mm, and the rates are at 1 and 4 μm/s respectively (`rtcm/RtcmSsrDecoder.cpp`).

The orbit record also carries a *reference point* for the corrected $\boldsymbol{r}^i$. Legacy RTCM SSR corrects to the satellite antenna phase centre (APC); IGS-SSR `4076` and the network product correct to the centre of mass (CoM) (`rtcm/RtcmSsrDecoder.cpp`). Under `realtime.honor_ssr_reference_point` (RTB-5; default OFF, byte-identical) the consumer records whether the applied overlay was APC-referenced (`ssr_orbit_apc_applied_` in `model/ModelGeometry.cpp`). `PppModel` then omits its own satellite-PCO term on that geometry, so the phase-centre offset already contained in the orbit correction is not counted twice.

1.2 Clock correction: refines $-c\,\mathrm{d}t^i$

The clock message carries a quadratic polynomial in the time since the reference epoch, optionally augmented by an additive high-rate delta δC_{hr}. The total satellite-clock correction, in metres, is

$$\Delta C^i(t) \;=\; C_0 + C_1\,\delta t + C_2\,\delta t^2 + \delta C_{\mathrm{hr}}, \qquad \delta t = t - t_{\mathrm{SSR}}, \tag{3}$$

converted to seconds and *added* to the baseline clock (`products/SsrCorrectionStore.cpp`):

$$\boxed{c\,\mathrm{d}t^i \;=\; c\,\mathrm{d}t^i_{\mathrm{base}} \;+\; \Delta C^i(t),} \tag{4}$$

where $\mathrm{d}t^i_{\mathrm{base}}$ is the precise CLK sample (post-processing) or the broadcast clock polynomial (live rooftop mode, `products/LiveEphemerisStore`). The term enters both undifferenced observation equations through $-c\,\mathrm{d}t^i$, identically on code and phase. The high-rate clock is added to the polynomial and does not replace it (`corrections/GnssCorrections.cpp`); it is subject to its own freshness test in the store. The C_2 field is a 27-bit two's-complement quantity at $2 \times 10^{-8}\,\mathrm{m/s^2}$. The time base of the polynomial is the record's content epoch (the provider's reference epoch), not its arrival time.

Split-cadence fast clocks. The stored clock can also move by a *fast tick* (`SsrCorrectionStore::pushFastClock`), which the network producer can run between full solves on decimated-out epochs. A fast tick rewrites three fields of an *existing* clock record: C_0 and the two time stamps (receive time and content epoch). Equation (3) is then evaluated at the fast δt with the C_1 and C_2 of the last full solve:

$$\Delta C^i(t) = C_0^{\mathrm{fast}} + C_1\,(t - t_{\mathrm{fast}}) + C_2\,(t - t_{\mathrm{fast}})^2. \tag{5}$$

Two fields are left untouched by contract, and both protect the consumer. The discontinuity counter is not advanced: a fast row continues the same clock arc, so it must not trigger the ambiguity reset of the discontinuity paragraph below at the fast cadence. The shared per-satellite SSR header is not restamped, so the code-bias, phase-bias, URA and slant-iono channels do not take the fast clock's age from a single refresh. The method only refreshes: when no clock record exists it returns false and creates nothing, so a consumer never receives a satellite that exists on the fast channel alone, without a bias set. It is called in-process (`filter/NetworkFastClock`) and on replay of an on-disk product (`products/NetworkProductIO`). It is default OFF (`network.fast_clock.enable`); the loader also disarms it when the run has no decimated-out epoch on which a tick could run.

1.3 IOD gate at correction-set boundaries

Orbit and clock corrections are deltas against a specific broadcast ephemeris, so they are applied only when the SSR record's IODE matches that of the broadcast ephemeris from which the baseline orbit/clock was evaluated:

$$\text{apply} \iff \big(\text{IODE}_{\text{brdc}} < 0\big) \ \vee \ \big(\text{IODE}_{\text{SSR}} < 0\big) \ \vee \ \big(\text{IODE}_{\text{brdc}} = \text{IODE}_{\text{SSR}}\big), \tag{6}$$

The gate is permissive when either side is unknown (`SsrCorrectionStore::iodeMatches`, `products/SsrCorrectionStore.hpp`). A combined orbit+clock record (RTCM 1060/1066, IGS-SSR sub-type 3) takes precedence over separate messages because it guarantees a single shared IODE (`products/SsrCorrectionStore.cpp`). The two overlays gate on separate anchors (RTF-2): the orbit delta on the record's orbit `iode` (`model/ModelGeometry.cpp`), the clock polynomial on a distinct `clock_iode` field (`corrections/GnssCorrections.cpp`). For a standard stream, `clock_iode` inherits the orbit anchor in `getOrbitClock`, so the two coincide and (6) is a single gate. Only a `brdc_rts` producer, which stamps its own clock IODE, makes them differ. At a correction-set boundary (the provider switches to a new ephemeris reference, hence a new IODE), the gate (6) declines the mismatched record until the matching broadcast ephemeris is available. This avoids a discontinuous jump in $\boldsymbol{r}^i$ or $c\,\mathrm{d}t^i$.

Records older than `max_age_s` (`processing.ssr_max_age_s`, default 30 s) are treated as absent. The age is that of the record's receive time relative to the query epoch, and the test is symmetric: a record dated more than the same limit ahead of the observation (a desynchronised live stream) is also rejected. The RTA-9(b) stale-hold ladder extends this binary clock gate to three rungs (`SsrCorrectionStore::classifyClockAge`). With `processing.ssr_clock_stale_hold_factor` $f > 1$, a clock aged in (`max_age_s`, $f \cdot$ `max_age_s`] is *StaleHeld*. It is still applied, extrapolated over the full age: the code evaluates the whole polynomial (3), of which $C_0 + C_1\,\delta t$ is the dominant part. The consuming rows receive a matching URA-staleness R inflation (`corrections/GnssCorrections.cpp`, `measurement/MeasurementModel.cpp`). A record beyond the hold window is *Rejected*. The default $f = 0$ reduces the ladder to the plain fresh-or-absent gate (byte-identical).

1.4 Satellite biases: refine $-B_s^i$ and $-b_s^i$

SSR delivers *observable-specific biases* (OSB): one value per (satellite, RINEX-3 signal) pair, in metres. They are applied as a known additive correction to the modelled observable, with no state and no Jacobian column (`measurement/MeasurementModel.cpp`). The code OSB refines B_s^i in the undifferenced code equation:

$$\boxed{P_{r,s}^i = \rho_r^i + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t_r^{(g)} + T_r^i + \mu_s I_{r,1}^i + B_{r,s} \underbrace{- B_s^i}_{\text{SSR OSB}} + \varepsilon^P,} \tag{7}$$

and the phase OSB refines b_s^i in the undifferenced phase equation:

$$\boxed{\Phi_{r,s}^i = \rho_r^i + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t_r^{(g)} + T_r^i - \mu_s I_{r,1}^i + \lambda_s N_{r,s}^i + b_{r,s} \underbrace{- b_s^i}_{\text{SSR OSB}} + \varepsilon^{\Phi}.} \tag{8}$$

The decoded value is the satellite bias as it is present in the observation (the underbraced term), so it is added to the clean model prediction with sign +1 (`kSsrCodeBiasSign` = `kSsrPhaseBiasSign` = +1, `measurement/MeasurementModel.cpp`). The sign can be confirmed only empirically against a provider's stream: float ambiguity fractions that cluster near one half instead of zero indicate an

inverted constant. The value enters the predicted observable as `ssr_bias_term` (`MeasurementModel.cpp`), summed with the receiver clock, troposphere, ionosphere, estimated code and phase biases and the ambiguity. Biases are queried per *raw* signal only: an ionosphere-free composite key such as `C1W_C5X` returns zero (code) or `nullopt` (phase) and is skipped, consistent with the uncombined parameterisation. The code OSB is a 14-bit field at 0.01 m, the phase OSB a 20-bit field at 0.1 mm (`rtcm/RtcmSsrDecoder.cpp`).

By default the application path has no age gate: `ssr_bias_term` is loaded whenever the store carries the exact RINEX signal key (or, for code only and with the optional band fallback of RTA-13 armed, a same-band sibling key; phase never falls back, to preserve the integer property). The optional A5 staleness gate (`processing.enable_ssr_bias_max_age`, default OFF) measures the age of the bias record against the observation epoch. Its limit is `ssr_bias_stale_hold_factor` × `processing.ssr_bias_max_age_s`; the factor defaults to 2 and the maximum age defaults to the clock maximum age, hence 30 s. Below that limit the last value stays applied at full weight across staleness gaps, so a provider update interval near the maximum age produces no model step; a bias older than the limit is dropped (`measurement/MeasurementModel.cpp`). The network-consumer profile arms the gate under bias consumption unless the configuration sets the key explicitly. The consequence of the application path for the state vector, avoiding a redundant estimated bias on the same row, is the subject of the next subsection.

1.5 Satellite-bias state gate and SSR code biases

The uncombined model can also *estimate* a per-(satellite, signal) code-bias state (`estimation.estimate_sat_biases`; see the state-vector section). The estimated state and a known SSR code OSB both enter the code row with unit coefficient, so allowing both for the same (i, s) is redundant: the stream already fixes the satellite code bias, and a free state collinear with it is nearly unobservable and absorbs noise. State allocation therefore consults both correction sources before creating a sat-bias state (`MeasurementModel::ensureRequiredStates`):

$$\text{create sat-bias}(i,s) \iff \neg\,\text{hasBIA}(i,s) \;\wedge\; \neg\,\text{hasSsrCB}(i,s) \;\wedge\; \text{pivotCovers}(i,s), \tag{9}$$

where hasBIA tests the static `.BIA` product (`PppModel::hasBiasEntry`) and hasSsrCB tests the live stream (`SsrCorrectionStore::hasCodeBias`, surfaced as `ssrCoversCodeSignal`). On a GLONASS FDMA signal that carries a per-slot inter-frequency-bias state, no sat-bias state is created either, because the two would be collinear. The gate rests on two design points:

- **Presence, not magnitude.** `hasCodeBias` tests whether the key is *present* in the per-signal map, which differs from `getCodeBias` $\neq 0$: a provider's present-but-zero entry still defines that signal's datum, so the estimator must not re-estimate it. The two queries agree by construction, including the optional same-band fallback, so a covered signal is never both estimated and SSR-corrected on one row. With the A5 staleness gate armed, the presence test uses the same age limit as the application path; otherwise it ignores age.
- **Per-signal, not global.** IGS streams are partial (for example, Galileo E1 covered and E5a/E5b not). The gate keys on the exact (i, s) pair. A signal the stream does not carry keeps its estimated state, its only available code correction, while a covered signal on the same satellite is left to the OSB. A strict per-epoch prune (`pruneSatBiasStatesExcept`) drops the state when SSR coverage arrives mid-arc and re-creates it (warm-restarted from the stashed cache) if coverage later disappears.

In post-processing the live store is empty, so hasSsrCB is always false and the gate reduces to the static-`.BIA` test.

Network-producer exemption. When the engine is the network estimator (`ProcessingMode::PPP_RTK_Network`), the roles are reversed: it *estimates* the satellite code and phase biases as states and emits them as a product (the Network mode document). Importing SSR biases there would count them twice and make those states unobservable. Both the `ssr_bias_term` application path (7) and the hasSsrCB branch of the state gate (9) are therefore disabled on the producer side (`ssrCoversCodeSignal` returns false). The producer's emitter (`products/NetworkProductEmitter.cpp`) pushes the estimated clocks and code and phase biases into a separate store instance, which closes the loop against a held-out user consumer.

Integer recoverability via consistent phase biases. Removing b_s^i restores the integer property of $N_{r,s}^i$ stated in the phase observation equation of the Foundations document. With the satellite phase bias known, the remaining non-integer part of the ambiguity is the receiver phase bias $b_{r,s}$ alone, which cancels under between-satellite single differencing $\Delta\cdot$ (the differencing section). The result is an integer single-difference ambiguity admissible to AR. This defines a *consistent* phase-bias product in operational terms: the provider must generate b_s^i on the same network datum whose integer property the rover then uses. The phase-bias message therefore carries three metadata fields per signal that AR depends on (`rtcm/RtcmTypes.hpp`): an integer-preservation indicator, a two-bit wide-lane integer indicator, and a four-bit *discontinuity counter*.

Discontinuity handling. An incremented discontinuity counter signals that the provider's phase bias jumped between correction sets, which breaks the integer connection of that arc. The engine treats the change as an ambiguity-arc boundary: the affected $N_{r,s}^i$ is reset before the Kalman update (`realtime/RealTimeEngine.cpp`, via `StateManager::noteSsrDiscontinuity`), so the post-jump phase is re-initialised instead of being fixed against a stale integer offset. Under the A5 staleness gate, a bias that changes between applied and dropped triggers the same reset. Two optional reactions, both default OFF, replace the reset: a phase-bias step translation (card PBT), which takes precedence, and a declared-step carry (card CC). Each shifts the ambiguity by the bias step where that step is known and falls back to the reset otherwise. A product whose phase biases are held by the producer is handled first (card PBH). The reset is the phase-bias counterpart of the orbit and clock IOD gate (6): both protect state continuity across correction-set boundaries, one by declining a mismatched delta, the other by resetting the dependent ambiguity.

1.6 GLONASS

GLONASS SSR is decoded and stored in the same way as the CDMA constellations. The legacy block (orbit 1063, clock 1064, code bias 1065, combined 1066, URA 1067, high-rate clock 1068, phase bias 1266) and the IGS-SSR `4076` subtypes in the GLONASS message-indicator band (IM = 41–47) both map onto the same orbit, clock and bias structures (`rtcm/RtcmSsrDecoder.cpp`). The orbit, clock and bias equations above apply unchanged; nothing in Equations (2), (4), (7) and (8) is constellation-specific.

The GLONASS-specific difference lies in the time tag, not in the correction mathematics. GLONASS SSR carries its content epoch as a time-of-day field (DF389, seconds since the start of the GLONASS UTC(SU)+3 h day) instead of a GPS time of week. The decoder converts it to the GPS time scale through `TimeUtils::glonassTodToGpsTime` (`rtcm/RtcmSsrDecoder.cpp`), resolving the day from the parser's last known GPS stream time. Without that conversion the SSR epoch would be $\{0, 0\}$; the store would then drop every GLONASS record as stale or IOD-mismatched (the `max_age_s` gate that accompanies (6)), and the satellite would fall back to broadcast grade without any error. After conversion, the per-record content epoch drives $\delta t = t - t_{\mathrm{SSR}}$ in Equations (2) and (4) as for

GPS. Whether the resulting GLONASS observable then *contributes* to a between-satellite-differenced solution is a separate estimator-side option (the FDMA per-channel receiver delay does not cancel in SD); see the differencing section.

1.7 Ionosphere: $\mu_s I^i_{r,1}$ off-wire, via the slant channel

The SSR standard defines a global VTEC spherical-harmonic message (IGS-SSR IM = 201) and regional STEC products, but this engine decodes neither. The IGS-SSR classifier rejects IM = 201 as unsupported (`rtcm/RtcmSsrDecoder.cpp`), so nothing on the RTCM wire writes the slant-iono state $I^i_{r,1}$. For a consumer of an RTCM stream, the ionospheric term $\mu_s I^i_{r,1}$ is therefore informed only by (i) the per-satellite iono *state* estimated by the filter and (ii) the optional GIM/IONEX deterministic prior (`atmosphere/GlobalIonosphereMap`, an independent VTEC→STEC mapping unrelated to the SSR stream).

The atmosphere loop is closed *off* the RTCM wire, through the same `SsrCorrectionStore` that carries the clock and bias overlays. The store has a dedicated undifferenced slant-iono channel (`SsrSlantIono`, via `SsrCorrectionStore::pushSlantIono` / `getSlantIono`). The network producer's emit seam (`filter/NetworkEmitStore`) evaluates its co-estimated SH-VTEC field (the co-estimated VTEC field) at the consumer's pierce points and pushes a per-LOS slant delay and 1σ. The consumer (`RealTimeEngine::processEpoch`, through its network ionosphere-prior step) folds each fresh record into an ionosphere-weighted (BLUP) pseudo-observation on the matching slant-iono state (`KalmanFilter::applyPseudoObservation`). This is the ionosphere-weighted constraint that makes single-epoch AR feasible. The fold is gated by `closed_loop.consume_network_iono` (default OFF: no rows are built and the update is byte-identical). It admits a value only when the producer refreshed it in the current epoch (`max_age` = $\frac{1}{2}$ the epoch spacing, or 30 s when the spacing is unknown), so a satellite the producer did not re-supply floats instead of inheriting a stale delay.

Three properties of the channel follow from the field and are not chosen by the consumer. The supply covers GPS and Galileo only: GLONASS is excluded from the SH field, so a consumer's GLONASS slant states float. When a second shell is armed, the pushed value is the *sum* over shells, each evaluated at its own pierce geometry, so the consumer does not depend on the layer model. The per-LOS σ is the field's representativeness floor (the SH representativeness sigma). Under `network.iono_supply_formal_sigma` (default OFF, byte-identical) it is root-sum-squared with the SH coefficients' *formal* σ, propagated through the same weighted basis, and the supply is skipped for a satellite that has no estimated satellite code bias (OSB) in the current producer solution.

On the consumer side the rows are not independent. They are folded as a per-constellation common-mode block with the noise structure of the GIM iono-weighting covariance. Frozen BSSD pivots, and any slant state with zero variance, are excluded for the reason given there: the common-mode block absorbs the datum offset of the *remaining* rows, so the datum satellite must not appear as a measurement.

A *standardised* on-wire STEC product remains open: the in-process channel is a closed-loop store hand-off, not an IGS-SSR or RTCM message that a consumer of a third-party stream can decode.

1.8 Provenance and code pointers

Both transports map onto the same internal correction structures (`rtcm/RtcmTypes.hpp`) through a single decoder (`rtcm/RtcmSsrDecoder.cpp`). Legacy RTCM uses per-constellation six-message blocks (orbit, clock, code bias, combined orbit+clock, URA, high-rate clock) and a separate phase-bias block (1265–1270). IGS-SSR `4076` repackages the same payloads under a message indicator

(IM). Its *tens* place ($\lfloor \mathrm{IM}/10 \rfloor$) selects the constellation (GPS $2x$, GLONASS $4x$, Galileo $6x$, QZSS $8x$, BeiDou $10x$, SBAS $12x$), and its *ones* place (IM mod 10) selects the subtype (1=orbit, 2=clock, 3=combined, 4=high-rate, 5=code bias, 6=phase bias, 7=URA). This subtype ordering differs from the legacy SSR block ordering, so the legacy mapping is not reused (`RtcmSsrDecoder.cpp`). VTEC (IM 201) and NavIC (IM 141–147) are rejected by the message-indicator classifier before the header is parsed, so they return `ok = false` with an empty header. For this reason the ionospheric term receives no *on-wire* SSR contribution (the off-wire `SsrSlantIono` store channel of the ionosphere subsection is unaffected).

The correction definitions Equations (2), (4) and (6) are implemented in the static, deterministic `applyOrbitToPosition` / `applyClockToBias` / `iodeMatches` members of `products/SsrCorrectionStore`, the interface this document describes. The stateful streaming layer (the mutex-guarded per-satellite cache with freshness gating on receive time, the `RealTimeEngine::feedRtcmFrame` push loop, and the obs↔SSR synchronisation that bounds how far the obs stream may run ahead of lagging corrections) is described in the Real-time engine document. It is kept separate so that the correction mathematics can be analysed and tested in isolation. The estimator-side warm-up gate `estimation.defer_until_corrections_ready` (`config/GnssConfigLoad.cpp`; default OFF, real-time only) defers estimation until a quorum of satellites is centimetre-grade from broadcast plus SSR (by default at least 5 in total and 4 per constellation). The filter then seeds code and phase together, instead of cold-starting on metre-grade broadcast clocks and fixing a contaminated ionosphere/bias/ambiguity partition.

The store also carries a per-epoch producer *heartbeat* (`NetworkProducerStatus`). It lets a consumer distinguish "the producer withheld this epoch" from "the producer stopped": a withheld epoch still writes its status line, although it publishes no corrections. The wire token set is forward-compatible: an unrecognised token from a newer producer maps to `State::Unknown` and is reported, never fatal (`SsrCorrectionStore::parseState` / `stateName`; both are static members of the producer-status structure above, declared in the store's header). One of the tokens reports a withhold driven by the health of the producer's own SH-VTEC field. The *policy* behind that withhold (what engages it, what it suppresses and what it leaves alone) is described in the Network mode document and is not restated here.

References

[1] International GNSS Service Real-Time Working Group. *IGS State Space Representation (SSR) Format, Version 1.00*. Format specification. RTCM message 4076 IGS-SSR sub-types. International GNSS Service (IGS), 2020.

[2] *RTCM Standard 10403.3: Differential GNSS Services – Version 3*. Special Committee No. 104; State Space Representation messages 1057–1068, 1240–1270, 4076. Radio Technical Commission for Maritime Services. Arlington, VA, USA, 2016.